CAPSICUMLABS

www.Capsicumlabs.com

BASECODE

Platform Development Requirements

Software Requirements Specification (SRS)

1. Document Information

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the Basecode platform developed by Capsicumlabs. The document serves as the authoritative reference for design, development, testing, and acceptance of the system.

1.2 Scope

Basecode is a lightweight, production-ready PHP 8.2 application framework designed to enable development teams to rapidly build secure and scalable business applications. The platform provides a pre-built foundation including:

- A RESTful API layer with JWT-based authentication and RBAC
- A browser-based Single Page Application (SPA) admin dashboard
- User and role management with fine-grained permission controls
- Comprehensive audit logging and event tracking
- Centralized system settings and branding configuration
- Background job processing via a queue system
- Extensible modular architecture for building domain-specific features

1.3 Intended Audience

Audience	Usage of this Document
Software Developers	Implementation guide for all platform features and APIs
QA / Test Engineers	Basis for test case design and acceptance criteria verification
Project Managers	Scope definition and delivery milestone planning
System Architects	Architecture validation and design compliance review
Stakeholders / Clients	Understanding platform capabilities and acceptance criteria

1.4 Document Conventions

> *Requirements are identified by unique IDs in the format: [MODULE-TYPE-NNN] where MODULE is the feature domain abbreviation, TYPE is FR (Functional), NFR (Non-Functional), or SEC (Security), and NNN is a three-digit sequence number.*

2. System Overview

2.1 Platform Description

Basecode is a custom PHP 8.2 framework built on Clean Architecture combined with Hexagonal (Ports & Adapters) architecture and Hierarchical Model-View-Controller (HMVC) modules. The framework does not depend on any third-party PHP framework such as Laravel or Symfony. It ships with its own dependency injection container, HTTP router, middleware pipeline, PDO-based database layer, CLI tool (artisan), and queue system.

2.2 Architecture Overview

The system is organized into four concentric architecture rings. Dependencies may only point inward — outer rings depend on inner rings, never the reverse. This constraint is enforced by a static analysis tool (deptrac).

Ring	Layer	Responsibilities
1	**Domain**	Entities, Value Objects, Ports (interfaces), Domain Events, Exceptions, Policies, Authorization definitions. No outward dependencies.
2	**Application**	Use Cases, Data Transfer Objects (DTOs), Queue and Scheduler runners. Depends on Domain only.
3	**Infrastructure**	Concrete adapters for database (PDO), cache, queue, mail, file storage, encryption, and logging. Implements Domain Ports.
4	**Presentation**	HTTP Router, Middleware Pipeline, API Controllers, Web handlers (login page, dashboard SPA). Depends on Application and Infrastructure.

2.3 Module Structure

Feature domains are organized as HMVC modules under the modules/ directory. Each module follows the same internal layer structure as the core source tree. The modules currently shipped with the platform are:

Module	Responsibility
User	User lifecycle management: authentication, CRUD, role assignment, status management
Role	Role-Based Access Control: role CRUD, permission matrix management
Audit	Audit event query API and background CSV export capability
AuditLog	Domain event listener infrastructure that writes auditable events to the database
Settings	Key-value system configuration store, SMTP mail settings, and branding management

Module	Responsibility
Example	Full-stack reference module providing a scaffold template for new module development

2.4 Technology Stack

Component	Technology / Specification
Server-side language	PHP 8.2+
Database	MySQL / MariaDB (utf8mb4)
Cache / Queue	Redis (production) or File-based (development)
Token library	firebase/php-jwt ^6.11
Mail library	phpmailer/phpmailer ^6.9
Frontend	Vanilla JavaScript SPA (admin.js + admin.css)
Web server	Apache with mod_rewrite (XAMPP-compatible), or any PHP-compatible server
Package management	Composer (PHP), npm (frontend assets)
Test framework	PHPUnit ^11.5
Architecture enforcement	deptrac (dependency direction rules)
CI/CD	GitHub Actions (.github/workflows/ci.yml)

3. Authentication & Session Management

3.1 Overview

The platform provides a secure login and session management system built on JSON Web Tokens (JWT). Tokens are cryptographically signed using HMAC-SHA256 (firebase/php-jwt library). The system enforces modern password storage standards and protects against common authentication attacks.

3.2 Functional Requirements

Req. ID	Category	Requirement
AUTH-FR-001	Login	The system shall provide a login endpoint accepting email and password credentials and returning a signed JWT access token and a refresh token on success.
AUTH-FR-002	Logout	The system shall provide a logout endpoint that revokes the current access token by adding it to the token blocklist and invalidates the associated refresh token.
AUTH-FR-003	Token Refresh	The system shall provide a token refresh endpoint that issues a new access token and rotates the refresh token using the provided valid refresh token.
AUTH-FR-004	Session Check	The system shall provide a session endpoint that returns the currently authenticated user's profile, role, and permissions based on a valid JWT.
AUTH-FR-005	Refresh Token Storage	Refresh tokens shall be stored in the database (refresh_tokens table) with per-token metadata including family ID, expiry, IP address, and user agent.
AUTH-FR-006	Token Rotation	Upon each successful token refresh, the old refresh token shall be marked as used and a new token issued within the same family. Detection of reuse of an already-used token shall revoke the entire token family.
AUTH-FR-007	Login Throttling	The system shall enforce rate-limiting on the login endpoint per IP address to prevent brute-force attacks. Configurable via cache-backed counters.
AUTH-FR-008	Password Rehash	During a successful login, the system shall opportunistically rehash the user's password to the current algorithm (Argon2id preferred, bcrypt accepted) if the stored hash is outdated.
AUTH-FR-009	Token Blocklist	Revoked access tokens shall be stored in the cache layer. All protected endpoints shall reject requests bearing blocklisted tokens even if the JWT signature and expiry remain valid.

Req. ID	Category	Requirement
AUTH-FR-010	Auth Transports	The system shall support two authentication transport modes: (a) Authorization: Bearer {token} HTTP header, and (b) a basecode_auth HTTP-only cookie.
AUTH-FR-011	Login Page	The system shall serve a branded HTML login page at the /login route with support for application logo and name from the branding settings.
AUTH-FR-012	Password Policy	The system shall reject authentication for any account whose stored password hash is not a recognized modern format (Argon2id or bcrypt). Legacy hashes (MD5, SHA-1, plaintext) shall never be accepted.

3.3 API Endpoints

Method	Endpoint	Auth Required	Description
POST	/api/auth/login	No	Authenticate user. Returns access token + refresh token.
POST	/api/auth/logout	Yes	Revoke current session and blocklist the access token.
POST	/api/auth/refresh	No	Exchange a valid refresh token for a new access token + rotated refresh token.
GET	/api/auth/session	Yes	Return the currently authenticated user's profile and permissions.
GET	/login	No	Render the branded HTML login page (web route).

3.4 Security Requirements

Req. ID	Category	Requirement
AUTH-SEC-001	JWT Configuration	JWT tokens shall be configured with: issuer (JWT_ISSUER), audience (JWT_AUDIENCE), TTL (JWT_TTL, default 3600 s), and clock skew tolerance (JWT_CLOCK_SKEW_SECONDS, default 30 s).
AUTH-SEC-002	Refresh Token TTL	Refresh tokens shall have a configurable TTL (REFRESH_TOKEN_TTL, default 1,209,600 s / 14 days) and shall be validated against this expiry on every use.
AUTH-SEC-003	Signing Key Strength	The JWT signing secret shall be resolved at runtime from environment configuration. Weak placeholder values shall be replaced by auto-generated runtime secrets in non-production environments. Production deployments shall require a strong configured secret.

Req. ID	Category	Requirement
AUTH-SEC-004	Auth Debug Logging	Authentication debug logging shall be disabled by default. It may only be enabled via an explicit environment flag (BASECODE_ENABLE_AUTH_DEBUG_LOG) in local development environments.

4. User Management

4.1 Overview

The User module provides a complete lifecycle management system for platform users. Users are linked to a single role that determines their permissions within the system. The module supports soft deletion, role assignment, and system account visibility controls.

4.2 User Entity

A user record contains the following attributes:

Field	Type	Description
id	INT UNSIGNED	Auto-increment primary key
name	VARCHAR(255)	Full display name of the user
email	VARCHAR(255)	Unique email address, validated via EmailAddress Value Object
password_hash	TEXT	Argon2id or bcrypt hash. Plaintext or legacy hashes are rejected.
status	VARCHAR(16)	One of: active, inactive, deleted
role_id	INT UNSIGNED	Foreign key to roles table. Cannot be null.
is_system_user	BOOLEAN	Marks internal/system accounts hidden from standard user listings
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Last update timestamp (auto-updated)
deleted_at	TIMESTAMP	Soft-delete timestamp. NULL means the record is active.
last_login_at	TIMESTAMP	Timestamp of the user's most recent successful login

4.3 Functional Requirements

Req. ID	Category	Requirement
USER-FR-001	List Users	The system shall return a paginated, filterable, searchable, and sortable list of users. Full-text search shall be supported via a dedicated search index on the users table.
USER-FR-002	Create User	Administrators shall be able to create a new user by supplying name, email, password, and role. Email shall be validated using the EmailAddress Value Object before persistence.

Req. ID	Category	Requirement
USER-FR-003	Get User Profile	The system shall return the complete profile of a user identified by ID, including name, email, status, role, and timestamps.
USER-FR-004	Update User Profile	Administrators shall be able to update a user's name, email, and other profile details. Email changes shall be re-validated.
USER-FR-005	Deactivate User	The system shall support deactivating a user (setting status to inactive) without deleting the record. Deactivated users cannot log in.
USER-FR-006	Restore User	The system shall support restoring a previously deactivated or soft-deleted user back to active status.
USER-FR-007	Delete User	The system shall support permanent hard deletion of a user record when explicitly requested. Soft-delete (status = deleted) shall also be supported as a reversible alternative.
USER-FR-008	Assign Role	Administrators shall be able to reassign a user's role to any existing role via a dedicated endpoint.
USER-FR-009	System Users	The system shall support marking accounts as system users. These accounts shall be hidden from standard user listings unless explicitly included via a flag. Visibility toggling shall be available via a dedicated endpoint.
USER-FR-010	Welcome Email	On user creation, the system shall optionally dispatch a welcome email to the new user. This behaviour shall be configurable at creation time.
USER-FR-011	Domain Events	User creation and other significant lifecycle events shall dispatch internal domain events that may be consumed by listeners (e.g., audit logging).

4.4 API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/users	Yes	List users with pagination, filtering, search, and sort.
POST	/api/users	Yes	Create a new user.
GET	/api/users/{id}	Yes	Get a single user's profile.
PATCH	/api/users/{id}	Yes	Update a user's profile details.
PATCH	/api/users/{id}/deactivate	Yes	Deactivate a user account.
PATCH	/api/users/{id}/restore	Yes	Restore a deactivated or soft-deleted user.
DELETE	/api/users/{id}	Yes	Permanently delete a user.

Method	Endpoint	Auth Required	Description
PATCH	/api/users/{id}/role	Yes	Assign a new role to the user.
PATCH	/api/users/{id}/system-user	Yes	Toggle system user visibility flag.
GET	/api/roles	Yes	List available roles for assignment (User module view).

5. Roles & Permissions (RBAC)

5.1 Overview

The platform implements Role-Based Access Control (RBAC). Each user is assigned exactly one role. A role carries a set of named permission strings. The permission middleware on each protected endpoint verifies that the authenticated user's role includes the required permission before granting access.

5.2 Role Entity

Field	Type	Description
id	INT UNSIGNED	Auto-increment primary key
name	VARCHAR(255)	Human-readable role name
slug	VARCHAR(255)	Unique machine-readable identifier used in JWT claims and permission resolution
description	TEXT	Optional plain-text description of the role's purpose
is_system	BOOLEAN	System roles are created during seeding and cannot be deleted
is_critical	BOOLEAN	Critical roles block deletion even when not marked as system roles
privilege_level	INT	Numeric privilege tier used for comparative access decisions
permissions	VARCHAR[]	Array of permission strings stored in the role_permissions join table
created_at	TIMESTAMP	Record creation timestamp

5.3 Permission Registry

All permission strings in the system are registered in a central PermissionRegistry. Permissions are grouped into domains. The registry is exposed via the /api/permissions endpoint, enabling the admin dashboard to dynamically build the permission assignment UI.

Domain	Permission Key	Description
User Management	users.view	View the user list and individual user profiles
	users.create	Create new user accounts
	users.update	Modify user profile details
	users.delete	Permanently delete user accounts

Domain	Permission Key	Description
	users.deactivate	Deactivate or restore user accounts
	users.assign-role	Assign or change a user's role
Role Management	roles.view	View role list and individual role details
	roles.create	Create new roles
	roles.update	Edit role name and description
	roles.updatePermissions	Modify the permission set assigned to a role
	roles.delete	Delete non-system, non-critical roles
Settings	settings.view	Read system settings values
	settings.update	Modify system settings values
	settings.mail.view	Read SMTP mail configuration
	settings.mail.update	Update SMTP mail configuration
Audit	audit.view	View audit event logs
	audit.export	Export audit logs to CSV
	audit.purge	Purge audit event records

5.4 Functional Requirements

Req. ID	Category	Requirement
ROLE-FR-001	List Roles	The system shall return a list of all roles with their associated permissions.
ROLE-FR-002	Get Role	The system shall return the full details of a single role including name, slug, description, privilege level, flags, and permissions.
ROLE-FR-003	Create Role	Administrators with the roles.create permission shall be able to create a new non-system role with a unique name and slug.
ROLE-FR-004	Update Role	Administrators with the roles.update permission shall be able to modify a role's name, slug, description, and privilege level.
ROLE-FR-005	Update Permissions	Administrators with the roles.updatePermissions permission shall be able to assign or remove any registered permissions from a role via a dedicated permissions update endpoint.
ROLE-FR-006	Delete Role	Administrators with the roles.delete permission shall be able to delete roles that are neither system roles nor critical roles and that have no users currently assigned.

Req. ID	Category	Requirement
ROLE-FR-007	Deletion Policy	The system shall enforce a RoleDeletionPolicy that blocks deletion of: (a) roles marked is_system = true, (b) roles marked is_critical = true, and (c) roles that still have at least one user assigned.
ROLE-FR-008	Super Admin Guard	The system shall ensure at least one active super-admin user exists at all times. The EnsureSuperAdminExistsUseCase shall be invoked at bootstrap to validate this invariant.
ROLE-FR-009	Permission Check	Super-admin users (identified by role id = 1 or role name containing both 'super' and 'admin', case-insensitive) shall automatically bypass all permission checks across all modules.
ROLE-FR-010	Role Details View	A dedicated details endpoint shall return an enriched role view combining role metadata with permission group information, intended for the permissions management UI.
ROLE-FR-011	Seeded Defaults	Default roles (including at minimum a super-admin role) and a corresponding system user shall be created automatically during initial database seeding.

5.5 API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/permissions	Yes	List all registered permission groups and their keys.
GET	/api/roles	Yes	List all roles with basic details.
GET	/api/roles/details	Yes	List roles with enriched permission group data for the management UI.
GET	/api/roles/{id}	Yes	Get full details for a single role.
POST	/api/roles	Yes	Create a new role.
PUT	/api/roles/{id}	Yes	Update a role's metadata.
PUT	/api/roles/{id}/permissions	Yes	Replace the complete permission set for a role.
DELETE	/api/roles/{id}	Yes	Delete a role (subject to deletion policy).

6. Audit Log & Insights

6.1 Overview

The platform automatically records security-relevant and operationally significant events for compliance, monitoring, and incident investigation. Audit logging is implemented as a two-module system: the AuditLog module captures and persists domain events as they are dispatched, while the Audit module provides query and export capabilities.

6.2 Automatically Tracked Events

The following domain events are captured by the AuditLog listener infrastructure and written to the audit_events table:

- User login (UserLoggedInEvent)
- User created (UserCreatedEvent)
- User profile updated
- User deactivated or restored
- User deleted
- Role created (RoleCreatedEvent)
- Role permissions updated (RolePermissionsUpdatedEvent)
- Settings updated
- Mail configuration updated (MailConfigUpdatedEvent)
- Any domain event implementing the AuditableDomainEvent interface

6.3 Functional Requirements

Req. ID	Category	Requirement
AUDIT-FR-001	Event Persistence	All auditable domain events shall be persisted to the audit_events table with: event type, actor user ID, target resource type and ID, timestamp, and a serialized detail payload.
AUDIT-FR-002	List Events	The system shall provide a paginated audit event listing endpoint supporting filters by date range, actor, event type, and target resource.
AUDIT-FR-003	Search Events	The system shall support resource-based search within the audit log to locate events related to a specific entity.
AUDIT-FR-004	Direct Export	Administrators with the audit.export permission shall be able to trigger a direct synchronous CSV export of audit events matching applied filters.
AUDIT-FR-005	Queued Export	For large result sets, the system shall support dispatching an audit.export.generate background job to the queue. The export file shall be written to storage and made available for download upon completion.

Req. ID	Category	Requirement
AUDIT-FR-006	Performance Indexes	The audit_events table shall be indexed for high-performance queries on: actor ID, event type, target resource, and timestamp. Composite indexes shall cover the most frequent query patterns.
AUDIT-FR-007	Insights UI	The admin dashboard Insights screen shall render an infinite-scroll audit log viewer with search, column customization, and filter controls.

6.4 API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/audit/events	Yes	List audit events with pagination and filters. Requires audit.view permission.
GET	/api/audit/export	Yes	Export audit events as CSV. Supports direct download or queued generation. Requires audit.export permission.

7. Settings & Configuration

7.1 Overview

The Settings module provides centralized, database-backed key-value configuration management. Settings are grouped by namespace prefix, support sensitivity marking for encrypted storage, and are exposed through a permission-protected API.

7.2 Settings Entity

Field	Type	Description
key	VARCHAR(255)	Unique dotted-namespace key (e.g. branding.app_name, app.timezone)
value	TEXT	Stored value. Sensitive values are encrypted using the OpenSSL adapter before storage.
is_sensitive	BOOLEAN	When true, the value is encrypted at rest and masked in API responses.
is_editable	BOOLEAN	When false, the setting is locked and cannot be modified via the API.
updated_at	TIMESTAMP	Last modification timestamp.
updated_by	INT UNSIGNED	Foreign key to users table. Records which administrator last changed this setting.

7.3 Settings Namespaces

Settings are organized by dotted key prefixes:

- branding.* — Application name, logo path, display mode (text or logo)
- app.* — General application configuration (timezone, locale, etc.)
- mail.* — SMTP connection parameters (host, port, username, password, sender)

7.4 Functional Requirements

Req. ID	Category	Requirement
SET-FR-001	Read Settings	The system shall return all readable settings to users with the settings.view permission. Sensitive settings shall have their values masked in responses.
SET-FR-002	Update Settings	Administrators with the settings.update permission shall be able to perform a bulk update of multiple non-locked settings in a single request.

Req. ID	Category	Requirement
SET-FR-003	Mail Configuration	The system shall provide dedicated endpoints to read and update SMTP mail configuration including host, port, username, password, encryption type, and sender address.
SET-FR-004	Test Mail	Administrators shall be able to send a test email to verify that the current SMTP configuration is functional. The response shall indicate success or failure with a descriptive message.
SET-FR-005	Branding — App Name	Administrators shall be able to configure the application display name stored under the branding.app_name setting key.
SET-FR-006	Branding — Logo	Administrators shall be able to upload an application logo image. The uploaded file path shall be cached in storage/cache/branding/logo-path.json for efficient retrieval during login page rendering.
SET-FR-007	Branding — Mode	Administrators shall be able to switch the branding display mode between text-based and logo-based modes.
SET-FR-008	Encryption	Settings marked is_sensitive = true shall be encrypted using the OpenSslEncryptionAdapter before storage and decrypted transparently on retrieval by the application layer.
SET-FR-009	Domain Events	Settings updates shall dispatch domain events (e.g. MailConfigUpdatedEvent) to enable dependent components to refresh configuration and to produce audit log entries.

7.5 API Endpoints

Method	Endpoint	Auth Required	Description
GET	/api/settings	Yes	Retrieve all settings. Sensitive values are masked. Requires settings.view.
PUT	/api/settings	Yes	Bulk update general settings. Requires settings.update.
GET	/api/settings/mail	Yes	Retrieve SMTP mail configuration. Requires settings.mail.view.
PUT	/api/settings/mail	Yes	Update SMTP mail configuration. Requires settings.mail.update.
POST	/api/settings/mail/test	Yes	Send a test email using current SMTP settings. Requires settings.mail.update.
POST	/api/settings/branding/logo	Yes	Upload branding logo image. Requires settings.update.

8. Admin Dashboard (Single Page Application)

8.1 Overview

The admin dashboard is a lightweight Single Page Application (SPA) built with Vanilla JavaScript (no framework dependency). It is served from /dashboard and communicates with the platform's REST API. The SPA is bundled during the build process and referenced from the HTML shell via a versioned asset manifest to support cache-busting in production deployments.

8.2 Dashboard Screens

Screen	Purpose	Key Features
Login	Entry point for authentication	Branded login form, error display, submit loading state
Dashboard	Main landing page after login	Summary cards, quick links, important settings overview
Users	User management interface	Paginated table, column selector, search and filter bar, create/edit forms, status actions
Roles	Role management and permission matrix	Role list, create/edit forms, interactive permission assignment matrix grouped by domain
Insights	Audit log viewer for compliance and monitoring	Infinite-scroll event list, search bar, date filter, event type filter, column customization
Configuration	Central hub for all administrative settings	Tabbed interface linking to Settings, Mail Setup, and Branding sub-screens
Settings	General application and branding configuration	Application name, logo upload, display mode toggle, key-value editor
Mail Setup	SMTP configuration and connectivity testing	SMTP field form, test email dispatch, success/error feedback

8.3 UI Features & Requirements

Req. ID	Category	Requirement
UI-FR-001	Dark / Light Mode	The dashboard shall support both dark mode and light mode. The active mode shall persist across sessions.
UI-FR-002	Font Scaling	The dashboard shall provide adjustable font size scaling to improve accessibility for users with visual preferences.

Req. ID	Category	Requirement
UI-FR-003	Collapsible Sidebar	The navigation sidebar shall be collapsible to maximize content area on smaller screens.
UI-FR-004	Breadcrumb Navigation	All non-root screens shall display breadcrumb navigation showing the user's location within the application hierarchy.
UI-FR-005	Toast Notifications	The application shall display non-blocking toast notifications for success, warning, and error outcomes of user actions.
UI-FR-006	Request Caching	The SPA shall cache API responses where appropriate (e.g., settings, roles list) to reduce redundant network requests.
UI-FR-007	JWT Expiry Monitoring	The SPA shall monitor the JWT access token expiry time and proactively refresh the token before expiry to maintain an uninterrupted session.
UI-FR-008	Inactivity Timeout	The SPA shall detect user inactivity and terminate the session after a configurable idle period, redirecting the user to the login page.
UI-FR-009	Permission-Based Navigation	Navigation menu items and UI controls shall be shown or hidden based on the currently authenticated user's permissions. Users shall not see controls for actions they are not permitted to perform.
UI-FR-010	Column Selector	The Users screen shall support a column visibility selector allowing administrators to show or hide specific table columns.
UI-FR-011	Infinite Scroll — Audit	The Insights audit log viewer shall implement infinite scrolling to load additional event records progressively as the user scrolls.
UI-FR-012	Permission Matrix	The Roles screen shall render an interactive permission matrix grouping all registered permissions by domain, with toggles to assign or remove individual permissions from a role.
UI-FR-013	Asset Versioning	In production environments, static assets (admin.js, admin.css) shall be referenced using versioned filenames from a build manifest to enable long-duration browser caching and automatic cache invalidation on deployment.

8.4 Web Routes

Method	Endpoint	Auth Required	Description
GET	/login	No	Render the branded HTML login page.
GET	/dashboard	Yes	Render the SPA shell HTML that bootstraps the admin dashboard.

9. Middleware & Security Pipeline

9.1 Overview

All HTTP requests pass through a composable middleware pipeline before reaching a controller. Middleware is applied in a defined order. Each layer may terminate the request early (e.g. rate limit exceeded, unauthorized) or pass it to the next handler. The pipeline is implemented using a MiddlewarePipeline class following the Chain of Responsibility pattern.

9.2 Middleware Stack

Middleware	Responsibility
ErrorHandlerMiddleware	Catches all uncaught exceptions from downstream handlers. Formats structured JSON error responses for API routes and HTML error pages for web routes. Maps domain exceptions to appropriate HTTP status codes.
RequestIdMiddleware	Generates or propagates a unique X-Request-ID header per request. Used for distributed tracing and log correlation.
HstsMiddleware	Injects Strict-Transport-Security header to enforce HTTPS connections in production environments.
CspMiddleware	Adds Content-Security-Policy headers to restrict the sources from which the browser may load resources, mitigating XSS attacks.
SecurityHeadersMiddleware	Sets defensive browser security headers: X-Frame-Options (clickjacking), X-Content-Type-Options (MIME sniffing), Referrer-Policy, and Permissions-Policy.
CorsMiddleware	Handles Cross-Origin Resource Sharing (CORS) with configurable allowed origins. Responds to preflight OPTIONS requests. Configurable via CORS_ALLOWED_ORIGINS environment variable or config/cors.php.
CsrfMiddleware	Validates CSRF tokens on non-idempotent web form submissions (POST, PUT, PATCH, DELETE on web routes) to prevent cross-site request forgery.
RateLimitMiddleware	Enforces per-IP sliding window rate limits. Default: 60 requests per 60 seconds. Configurable per path prefix. Returns HTTP 429 with Retry-After header when exceeded. Uses cache backend (File or Redis).
AuthMiddleware	Extracts and verifies the JWT access token from Authorization header or cookie. Attaches an AuthUserDTO to the request context. Returns HTTP 401 for missing or invalid tokens.
PermissionMiddleware	Verifies that the authenticated user holds the permission string required by the current route. Super-admin users bypass this check. Returns HTTP 403 if the required permission is absent.

9.3 Security Requirements

Req. ID	Category	Requirement
SEC-NFR-001	Inward Dependencies	Architecture dependency direction shall be enforced by deptrac. CI pipeline builds shall fail if any outward dependency violation is detected.
SEC-NFR-002	HTTPS Enforcement	The HSTS middleware shall set a max-age of at least one year on production deployments.
SEC-NFR-003	Trusted Proxies	The system shall only trust X-Forwarded-For headers from IP addresses or CIDR ranges explicitly listed in the TRUSTED_PROXIES environment variable.
SEC-NFR-004	Debug Mode	APP_DEBUG=true shall be rejected in production and staging environments.
SEC-NFR-005	Bootstrap Seeders	Automatic database seeding on request handling shall be disabled by default (BASECODE_ENABLE_BOOTSTRAP_SEEDERS=false). Seeding shall only occur via explicit CLI commands.

10. Queue & Background Processing

10.1 Overview

The platform includes a built-in job queue for deferring time-consuming or unreliable operations (email sending, large export generation) out of the HTTP request cycle. The queue system supports two drivers and provides dead-letter storage for failed jobs.

10.2 Queue Drivers

Driver	Description & Applicability
Redis	Required in production and staging environments. Uses Redis lists with a configurable key prefix and dead-letter key. Enforced automatically unless BASECODE_ALLOW_FILE_CACHE_IN_PRODUCTION is set.
File	Used in local and development environments. Persists jobs as JSON in storage/queue/. Uses lock files for safe concurrent access. Failed jobs written to storage/queue/failed.json.

10.3 Functional Requirements

Req. ID	Category	Requirement
QUEUE-FR-001	Job Dispatch	The system shall provide a QueuePort interface for dispatching QueuedJobDTO payloads. Use cases shall depend on the port, not the concrete driver.
QUEUE-FR-002	Worker Process	The platform shall provide a QueueWorker that processes jobs one at a time, invoking registered handler functions by job name.
QUEUE-FR-003	CLI Command	Queue worker shall be startable via: php artisan queue:work
QUEUE-FR-004	Retry Limit	Each job shall track attempt count. Jobs exceeding the configured max_attempts (default: 5) shall be moved to the dead-letter storage and not requeued.
QUEUE-FR-005	Visibility Timeout	The queue shall enforce a configurable visibility_timeout_seconds (default: 60 s) to prevent a job from being permanently stuck if a worker crashes mid-processing.
QUEUE-FR-006	Built-in Job: Mail	The mail.send job shall invoke the mailer adapter to send a message from the queue, decoupling mail delivery from synchronous HTTP responses.

Req. ID	Category	Requirement
QUEUE-FR-007	Built-in Job: Export	The audit.export.generate job shall invoke the audit export use case to produce a CSV file in the storage/reports/ directory.
QUEUE-FR-008	Dead Letter	Failed jobs beyond the retry limit shall be recorded in a dead-letter store (file or Redis key) for manual inspection and replay.

11. Cache System

The cache system abstracts storage of short-lived data behind the CachePort interface. Three adapters are provided:

Adapter	Description
RedisCache	Production adapter. Connects to Redis using configured host/port/db. All keys are namespaced with a configurable prefix.
FileCache	Development / shared-hosting adapter. Persists serialized cache entries to storage/cache/ with .lock files for write safety. SHA-256 filenames prevent filesystem conflicts.
InMemoryCache	Test adapter. Stores entries in a PHP array for the duration of the current process. No persistence.

The cache is consumed by:

- Rate limiting (per-IP sliding window counters)
- JWT token blocklist (revoked token storage)
- Route cache (serialized API route table to avoid re-evaluation on every request)
- Login attempt throttling (brute-force protection counters)

12. Mail System

Mail delivery is abstracted behind the MailerPort interface. Four adapters are available:

Adapter	Description
SmtpMailer	Production adapter using PHPMailer. Reads SMTP configuration (host, port, credentials, encryption) from the Settings module at send time.
QueuedMailer	Wraps any mailer adapter and enqueues a mail.send job instead of delivering inline. Decouples mail delivery from the HTTP request cycle.
LogMailer	Writes mail content and metadata to storage/logs/ instead of sending. Used in staging or debug environments for safe outbound mail interception.
NullMailer	Silently discards all mail. Used in automated test suites to prevent real email delivery.

Built-in email templates (extending BaseHtmlEmailTemplate):

- WelcomeUserTemplate — sent to newly created users
- TestEmailTemplate — sent when testing SMTP configuration from the Settings screen

13. Observability & Diagnostics

13.1 Structured Logging

The platform uses a MonologAdapter behind the LoggerPort interface to write structured JSON log entries. Log streams are separated by concern:

Log File	Content
storage/logs/app.jsonl	General application events, request lifecycle, errors
storage/logs/queue.jsonl	Queue worker job processing events
storage/logs/security.jsonl	Authentication events, permission violations, rate limit breaches
storage/logs/queue-maintenance.log	Queue maintenance ping handler output

13.2 Metrics

The FileMetricsAdapter emits labeled counters to storage/metrics/metrics.json in Prometheus-compatible label format (e.g. http.requests.total|method=GET,path=/api/users). Tracked metrics include:

- HTTP request counts by method and path
- HTTP response counts by path, status code, and bucket (2xx, 4xx, 5xx)
- Queue job processing counts by job name and queue

> *In production environments the metrics driver defaults to noop (disabled) unless explicitly set via the METRICS_DRIVER environment variable.*

13.3 Performance Diagnostics

The platform provides two built-in performance diagnostic tools:

- Performance Probe Endpoint: GET /api/perf/probe returns a lightweight health and latency check response for monitoring systems.
- DB EXPLAIN Report: php artisan db:explain:top identifies the highest-traffic endpoints, runs MySQL EXPLAIN on their primary queries, and writes formatted markdown reports to storage/reports/. Supports slow query log analysis for index optimization.

14. Database & Migrations

14.1 Database Schema

The following core tables are created by the initial setup script (database/setup.sql):

Table	Purpose
migrations	Tracks applied migration versions to prevent re-application. Enforces uniqueness on (version, name).
roles	Role definitions with name, slug, description, privilege level, and system/critical flags.
role_permissions	Join table mapping role_id to individual permission strings. Composite PK on (role_id, permission). Cascades on role deletion.
users	User accounts with status lifecycle, soft delete support, role FK, system user flag, and login timestamp.
refresh_tokens	Refresh token store with family ID for rotation chain tracking, IP/UA metadata, expiry, and revocation timestamps.
settings	Key-value configuration store with sensitivity and editability flags, and updated_by FK to users.
examples	Reference table for the Example scaffold module.

14.2 Migration System

Req. ID	Category	Requirement
DB-FR-001	Versioned Migrations	All schema changes shall be implemented as versioned migration files following the naming convention: YYYYMMDD_description.up.sql / .down.sql stored in database/migrations/.
DB-FR-002	Migration Runner	The platform shall provide a MigrationRunner that applies unapplied migrations in order and records each applied migration in the migrations table.
DB-FR-003	Rollback Support	Each migration shall have a corresponding .down.sql rollback script to reverse the schema change.
DB-FR-004	Idempotent Setup	The database/setup.sql script shall use CREATE TABLE IF NOT EXISTS to be safely re-runnable.
DB-FR-005	Runtime Bootstrap	A RuntimeSchemaBootstrap mechanism shall ensure the base schema exists on first run. This feature shall be disabled by default in production and only enabled for bootstrap workflows via explicit environment flag (RUNTIME_SCHEMA_BOOTSTRAP).

15. Developer Tooling & Extensibility

15.1 CLI (Artisan)

The platform ships with a CLI tool (artisan) providing the following commands:

Command	Description
php artisan queue:work	Start the queue worker process
php artisan db:explain:top	Run EXPLAIN analysis on top HTTP endpoints and write markdown reports
php artisan seed:bootstrap	Run initial database seeders (roles, system user, settings)
php artisan migrate	Apply pending database migrations
composer test	Run PHPUnit test suite with colored output
npm run build:assets	Build and version-stamp frontend assets (admin.js, admin.css)

15.2 Example Module (Scaffold Reference)

The modules/Example directory provides a complete reference implementation of a feature module. It demonstrates the full vertical slice from Domain to Presentation and should be used as the scaffold template when creating new modules. It includes:

- Domain: ExampleEntity, ExampleIdentifier (Value Object), ExampleRepositoryPort, ExampleCreatedEvent
- Application: Full CRUD Use Cases (Create, Get, List, Update, Delete), all DTOs (Input and Output)
- Infrastructure: PdoExampleRepository (concrete PDO implementation of the port)
- Presentation: ExampleApiController, ExampleController (web), Blade view templates, route definitions

15.3 API Versioning

All API routes registered under /api/* are automatically mirrored as /api/v1/* by the route loader. This provides zero-cost forward compatibility, allowing client applications to migrate to versioned URLs without any backend changes.

> *Explicitly versioned routes (/api/v1/*) are not duplicated — only unversioned /api/* routes receive a v1 mirror.*

15.4 Dependency Injection Container

The platform ships with a lightweight DI container (Container.php) that supports interface-to-implementation binding, lazy instantiation, and shared singleton instances. Module bindings are registered in config/bindings.php and centralized in src/Infrastructure/Bootstrap/ContainerBindings.php.

16. Non-Functional Requirements

16.1 Performance

Req. ID	Category	Requirement
PERF-NFR-001	Route Caching	Compiled API routes shall be cached to storage/cache/routes/ to eliminate route file re-evaluation on each request. The cache shall be invalidated automatically when any route file is modified.
PERF-NFR-002	OPcache Preload	A preload configuration (config/opcache.preload.php) shall be provided for PHP OPcache to preload commonly used classes at server startup, reducing per-request compilation overhead.
PERF-NFR-003	DB Indexing	All frequently queried columns (user status, role FK, soft-delete timestamp, refresh token expiry, audit event indexes) shall have appropriate database indexes.
PERF-NFR-004	Queue for Heavy Ops	Any operation with unpredictable latency (outbound email, large file generation) shall be offloaded to the queue worker rather than blocking the HTTP response.

16.2 Security

Req. ID	Category	Requirement
SEC-NFR-006	Password Hashing	All user passwords shall be stored as Argon2id or bcrypt hashes. The system shall never store or accept MD5, SHA-1, or plaintext passwords.
SEC-NFR-007	Sensitive Settings	Settings marked is_sensitive shall be encrypted at rest using AES-256-CBC via the OpenSslEncryptionAdapter before database storage.
SEC-NFR-008	Token Security	JWT signing secrets below a minimum entropy threshold shall be automatically replaced by generated runtime secrets on non-production environments. Production shall enforce explicit configuration.
SEC-NFR-009	XSS / CSRF	The CSP middleware and CSRF middleware shall be active on all web routes to prevent cross-site scripting and cross-site request forgery.
SEC-NFR-010	SQL Injection	All database queries shall use parameterized PDO statements. No user-supplied input shall be interpolated directly into SQL strings.

16.3 Maintainability

Req. ID	Category	Requirement
MAINT-NFR-001	Architecture Compliance	The deptrac tool shall be integrated into the CI pipeline. Any commit introducing an architecture dependency violation shall fail the build.
MAINT-NFR-002	Test Coverage	The test suite shall cover all Use Cases, Middleware, Infrastructure adapters (JWT, Cache, PDO, Mail), the DI Container, and the Policy Registry. Unit tests shall be independent of external services.
MAINT-NFR-003	Module Isolation	Each module under modules/ shall be independently deployable. A module shall not directly import classes from another module's internal layers.
MAINT-NFR-004	Env Configuration	All environment-specific values shall be externalized to .env files. The .env.example file shall document every supported configuration variable.

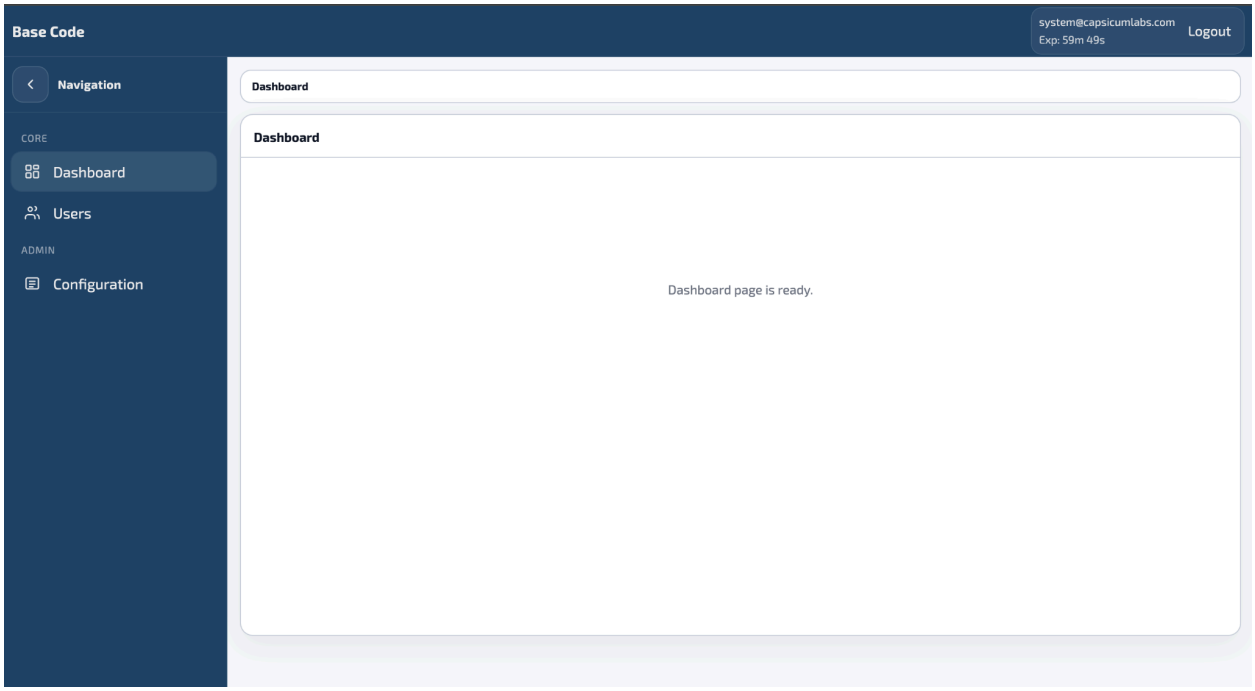
16.4 Deployment

Req. ID	Category	Requirement
DEPLOY-NFR-001	Apache Compatibility	The platform shall ship with .htaccess files at both the project root and the public/ directory to route all requests through the public/index.php front controller.
DEPLOY-NFR-002	Environment Guards	The system shall enforce production/staging environment restrictions including: Redis-only queue, no APP_DEBUG, no legacy file cache unless explicitly opted-in.
DEPLOY-NFR-003	Docker Ready	A docker/ directory shall be provided as the basis for containerized deployment. Docker configuration details are outside the scope of this document.
DEPLOY-NFR-004	CI/CD	A GitHub Actions workflow (.github/workflows/ci.yml) shall run the full PHPUnit test suite on every push to the repository.

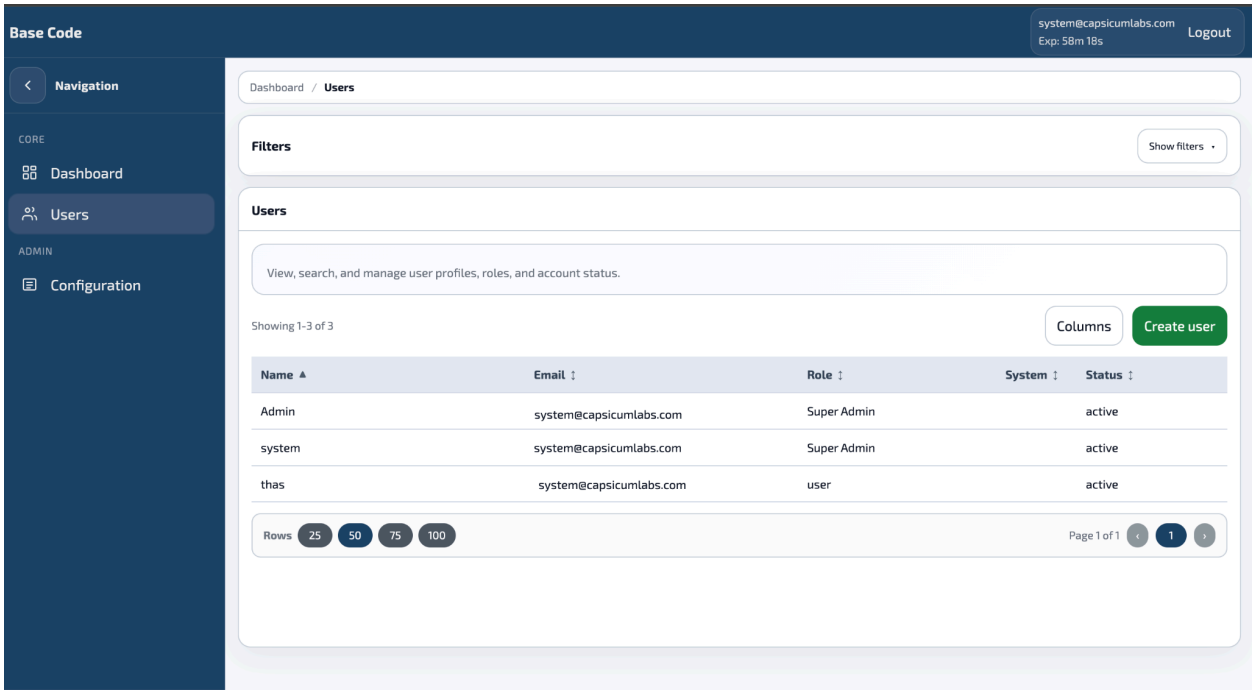
17. UI

1. Login

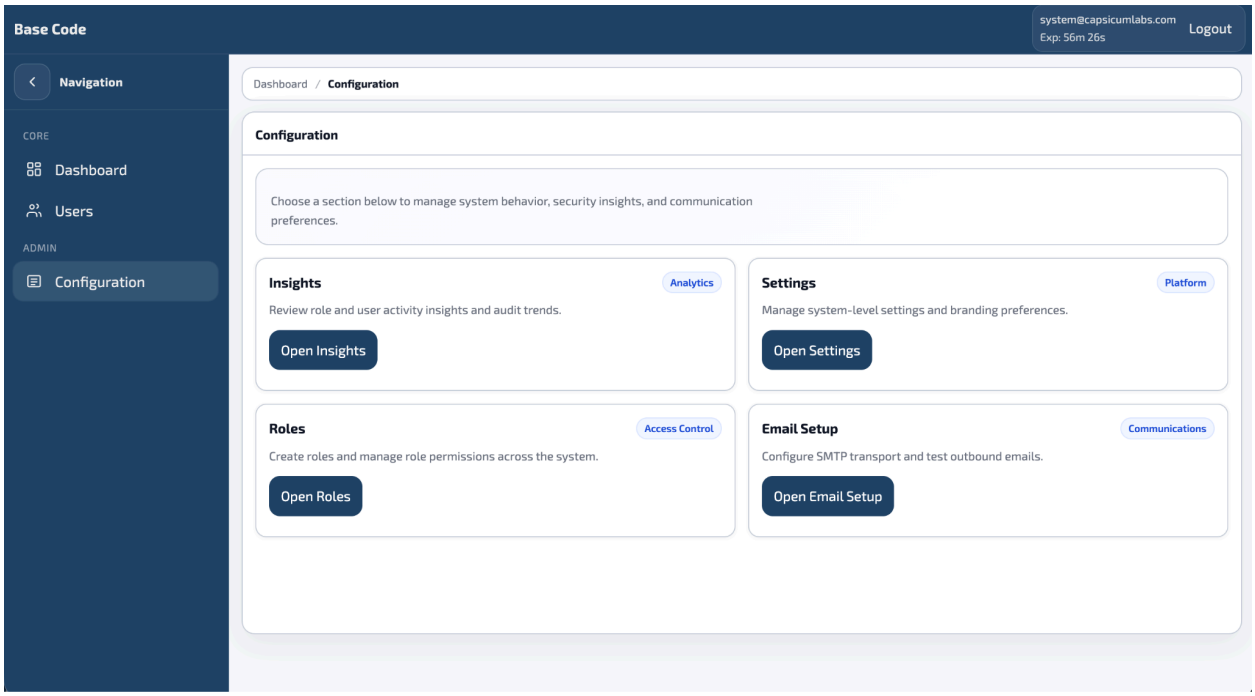
2. Dashboard



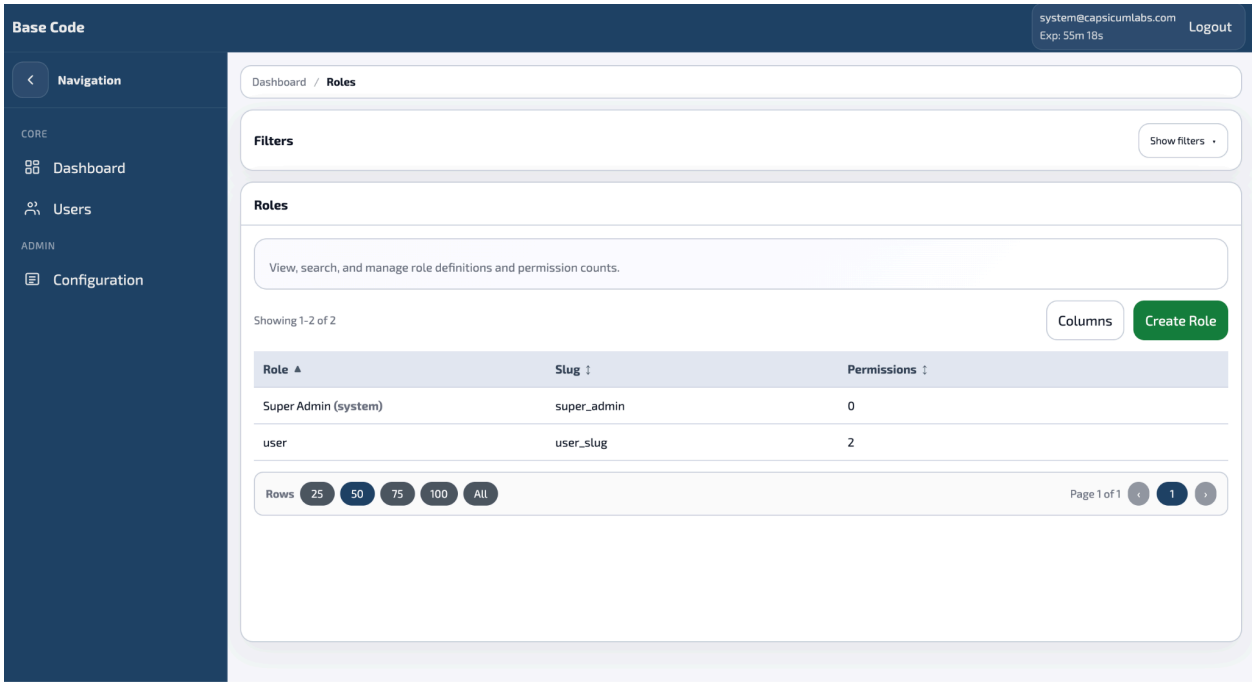
3. Users

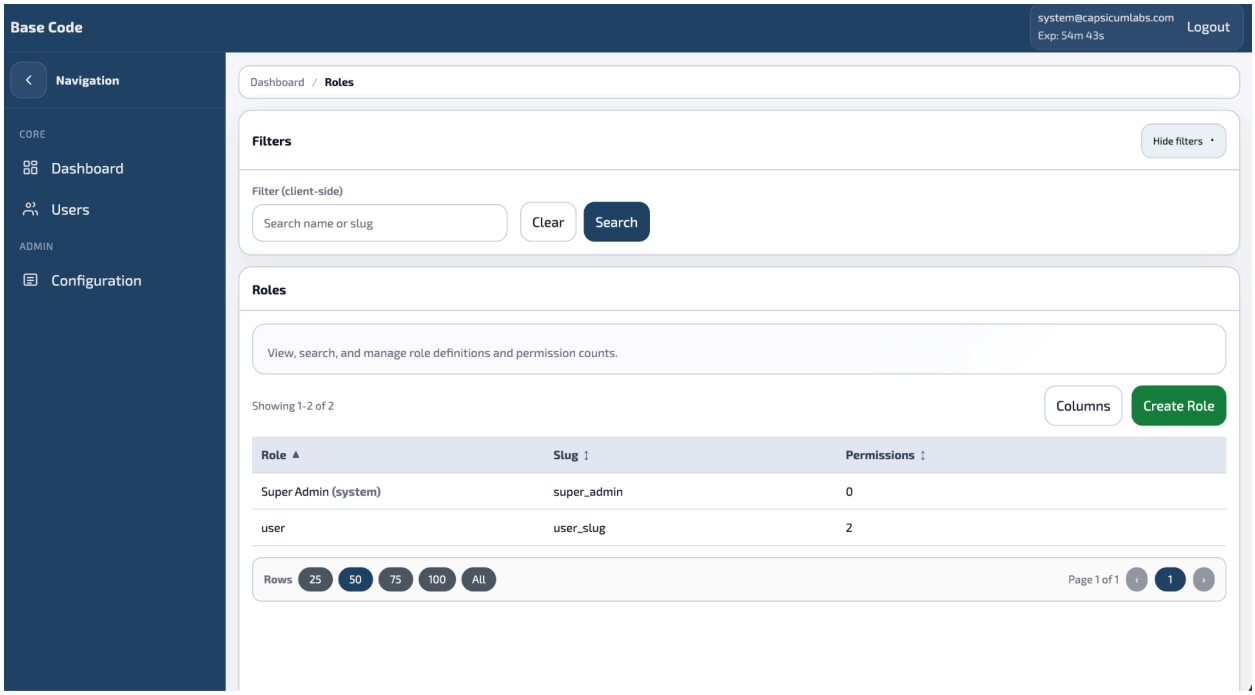


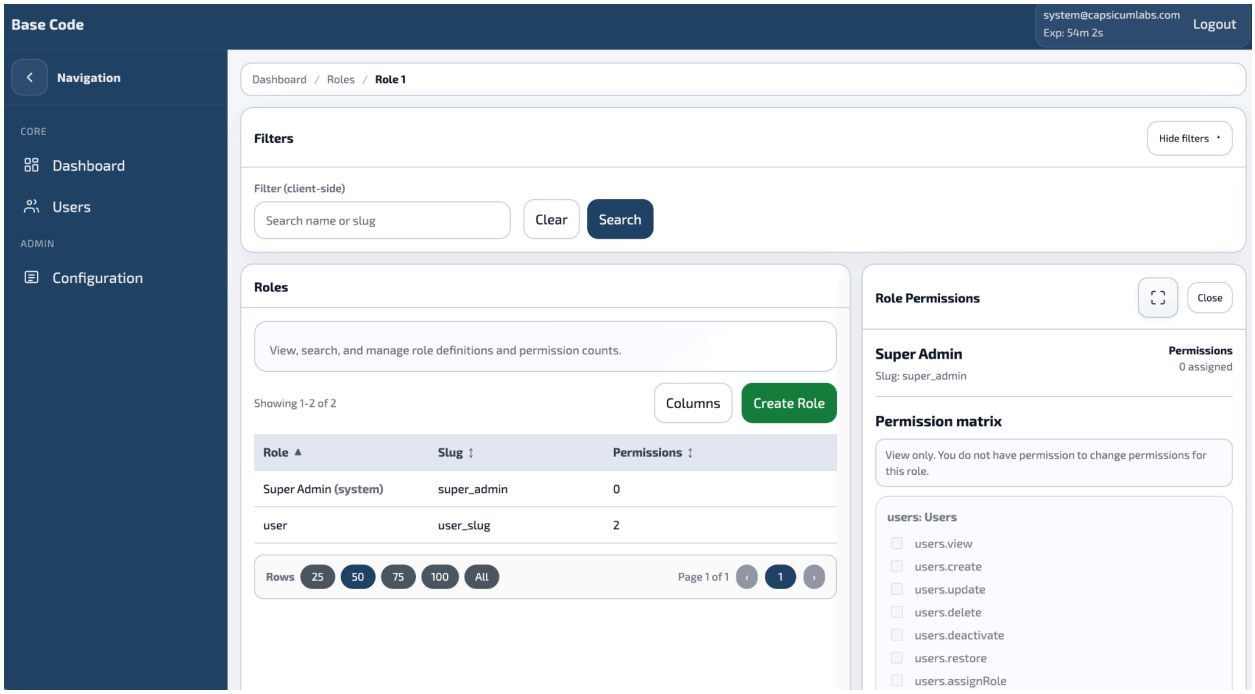
4. Config

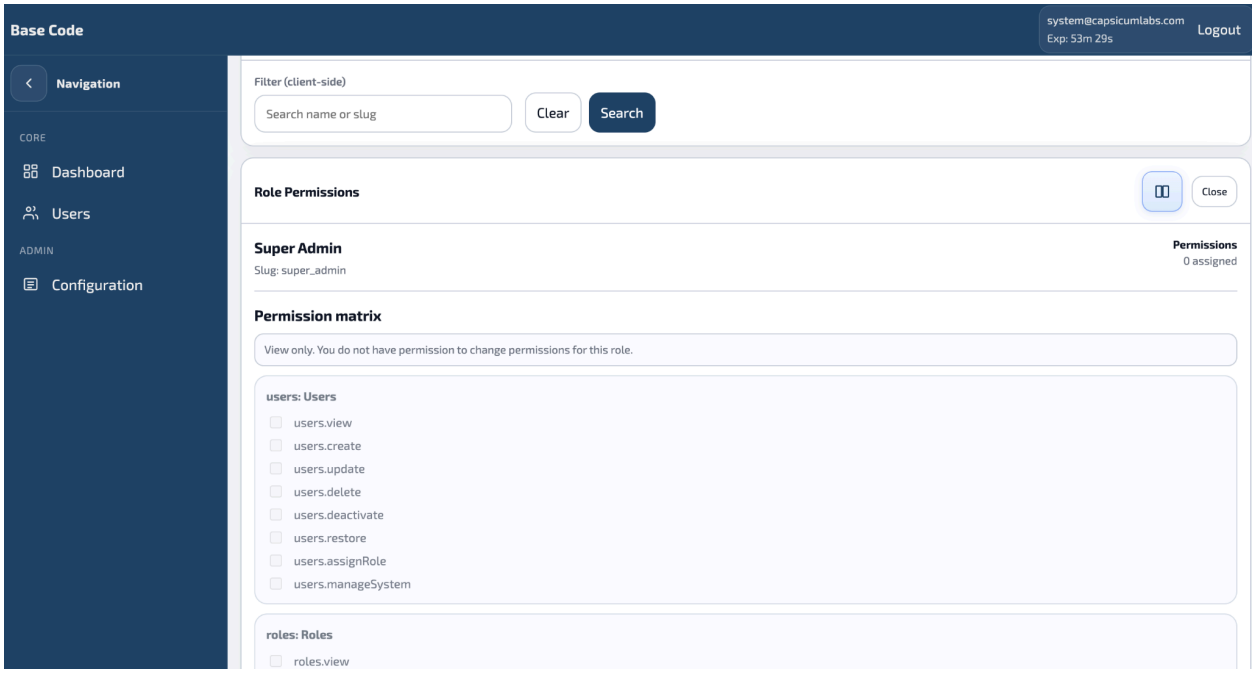


5. Role


Base Code
system@capsicumlabs.com
Exp: 55m 18s
Logout
Navigation
CORE
Dashboard
Users
ADMIN
Configuration
Dashboard / Roles
Filters
Show filters
Roles
View, search, and manage role definitions and permission counts.
Showing 1-2 of 2
Columns
Create Role
Role
Slug
Permissions
Super Admin (system)
super_admin
0
user
user_slug
2
Rows 25 50 75 100 All
Page 1 of 1
1


Base Code
system@capsicumlabs.com
Exp: 54m 43s
Logout
Navigation
CORE
Dashboard
Users
ADMIN
Configuration
Dashboard / Roles
Filters
Hide filters
Filter (client-side)
Search name or slug
Clear
Search
Roles
View, search, and manage role definitions and permission counts.
Showing 1-2 of 2
Columns
Create Role
Role
Slug
Permissions
Super Admin (system)
super_admin
0
user
user_slug
2
Rows 25 50 75 100 All
Page 1 of 1
1


Base Code
system@capsicumlabs.com
Exp: 54m 2s
Logout
Navigation
CORE
Dashboard
Users
ADMIN
Configuration
Dashboard / Roles / Role 1
Filters
Hide filters
Filter (client-side)
Search name or slug
Clear
Search
Roles
View, search, and manage role definitions and permission counts.
Showing 1-2 of 2
Columns
Create Role
Role
Slug
Permissions
Super Admin (system)
super_admin
0
user
user_slug
2
Rows 25 50 75 100 All
Page 1 of 1
1
Role Permissions
Close
Super Admin
Slug: super_admin
Permissions
0 assigned
Permission matrix
View only. You do not have permission to change permissions for this role.
users: Users
users.view
users.create
users.update
users.delete
users.deactivate
users.restore
users.assignRole


Base Code
system@capsicumlabs.com
Exp: 53m 29s
Logout
Navigation
CORE
Dashboard
Users
ADMIN
Configuration
Filter (client-side)
Search name or slug
Clear
Search
Role Permissions
Close
Super Admin
Slug: super_admin
Permissions
0 assigned
Permission matrix
View only. You do not have permission to change permissions for this role.
users: Users
users.view
users.create
users.update
users.delete
users.deactivate
users.restore
users.assignRole
users.manageSystem
roles: Roles
roles.view

18. Glossary

Term	Definition
RBAC	Role-Based Access Control. A security model where access permissions are assigned to roles, and users are assigned roles.
JWT	JSON Web Token. A signed, self-contained token format used for stateless authentication.
SPA	Single Page Application. A web application that loads a single HTML page and dynamically updates content via JavaScript.
HMVC	Hierarchical Model-View-Controller. An architectural pattern where feature domains are organized as self-contained modules, each with their own MVC layers.
Port	In Hexagonal Architecture, an interface in the Domain layer that defines a contract for an external capability (e.g. a database, cache, or mail service).
Adapter	A concrete implementation of a Port in the Infrastructure layer. Adapters may be swapped without changing Domain or Application code.
Use Case	An Application-layer class that orchestrates domain logic to fulfill a single business operation (e.g. CreateUserUseCase).
DTO	Data Transfer Object. A plain object used to carry data across layer boundaries without exposing domain internals.
Domain Event	A message dispatched within the domain to signal that a significant state change has occurred. Consumed by listeners for side effects such as audit logging.
Soft Delete	A deletion pattern where a record is marked as deleted (via deleted_at timestamp) rather than physically removed from the database, enabling later restoration.
Token Family	A group of related refresh tokens issued through a chain of rotations from an original token. Detecting reuse of any token in the family revokes all tokens in that family.
Dead Letter	Storage for jobs that have exceeded their maximum retry attempts and cannot be successfully processed. Allows for manual inspection and replay.
DXA	Document XML Attribute units. The unit system used by DOCX/Office Open XML, where 1 inch = 1,440 DXA.
DI Container	Dependency Injection Container. A component that manages object creation and wires dependencies according to registered bindings.
SRS	Software Requirements Specification. A document describing the intended capabilities, appearance, and interactions of a software system.